

CSCE 638 NATURAL LANGUAGE PROCESSING · MODULE 08 · LECTURE NOTES

Adaptation

Getting a pretrained model to do your task.

MODULE OUTCOMES

- Explain the adaptation methods and their costs.
- Explain parameter-efficient fine-tuning.
- Explain in-context learning and its limits.
- Explain instruction tuning and preference training.
- Choose a method against real constraints.

1 The spectrum

Method	What changes	When it fits
Prompting	Nothing — only the input text.	No training data at all; fast iteration; prototyping.
Few-shot prompting	Nothing; examples are placed in the context (§3).	A handful of examples and no training budget.
Linear probe	A classifier trained on frozen features.	Small data; and a strong, cheap baseline — see the note.
Parameter-efficient fine-tuning	A small set of added or modified weights (§2).	The usual answer for most real tasks.
Full fine-tuning	Every weight in the model.	Ample labelled data and compute, and a substantial domain shift.
Continued pretraining	Every weight, on raw domain text with the pretraining objective.	A genuinely different domain with a lot of unlabelled text — clinical or legal corpora, or code.

RUN THE CHEAPEST METHOD FIRST, BECAUSE IT TELLS YOU WHAT THE TASK NEEDS

A linear probe on frozen features takes minutes — extract the representations once, fit logistic regression — and **if it works, the pretrained representations already contain the signal and you are finished.**

If it does not work, that is informative rather than disappointing: either the task requires information the representations do not carry, or it requires a nonlinear decision boundary over them — and **you can distinguish those two by trying a small nonlinear head**, which takes another ten minutes.

Whereas starting with full fine-tuning gives you a number and no diagnosis — **you do not know whether the gain came from task adaptation or simply from the additional effective capacity**, and you cannot tell what a cheaper method would have achieved.

Which is CSCE 633 Module 04's ablation discipline arriving as a practical workflow — **the cheap method is simultaneously a baseline and an experiment**, and skipping it forfeits both. **Module 10 §2 then requires the baseline anyway**, so running it first costs nothing.

2 Parameter-efficient methods

INSTEAD OF updating W ($d \times d$), learn
 $W' = W + BA$
 where B is $d \times r$ and A is $r \times d$, with $r \ll d$.

SO the trainable parameters go from d^2 to $2dr$ --
 frequently under 1% of the model's total.

WHY IT WORKS
 the weight UPDATE needed for a task appears to be
 approximately low rank, even though the weights
 themselves are not. Adaptation is a small
 correction to an existing function, not the
 learning of a new one.

WHAT IT BUYS
 trains in much less memory, since the optimiser
 state scales with the TRAINABLE parameters
 rather than with the total
 many task adapters can share one base model in
 memory, which changed how these are deployed
 and BA can be merged into W at inference time, so
 there is no added latency at all

AND THE QUALITY IS CLOSE TO FULL FINE-TUNING on most
 tasks -- which is the empirical result that made it
 the default.

The low-rank-update hypothesis is the actual insight, and it is worth stating as a claim about adaptation: **a pretrained model already computes approximately the right function, and a task requires a small correction rather than a different function** — which is Module 07 §3's transfer argument expressed in

terms of weights. **And the merge-at-inference property is what makes it free in production**, while **the shared-base-model property is what changed deployment**: serving a hundred fine-tuned variants used to mean a hundred models.

3 In-context learning

FEW-SHOT PROMPTING CHANGES THE MODEL'S BEHAVIOUR WITHOUT CHANGING THE MODEL

Examples placed in the context condition the prediction — the weights are entirely unchanged, so nothing is retained between requests, which is why 'learning' is a somewhat misleading word for what is happening.

And the examples do two distinct things: they specify the task format, and they supply some information about the input-output mapping — and the evidence is that the format contribution is substantially the larger of the two, since replacing the labels with random ones degrades performance much less than removing the examples does.

Which is precisely why the results are sensitive to things that ought not to matter: the order of the examples, the exact wording of the labels, the formatting, and the presence of a trailing space all move the output measurably (Module 02 §3's tokenisation sensitivity contributing).

So treat it as a fast, high-variance method. Excellent for prototyping and for tasks where no training data exists, and a poor choice where consistency matters — and Module 10 asks you to measure that variance rather than assume it away.

- Context costs money and latency on every single request, forever — whereas fine-tuning pays the cost once, which inverts the economics above a modest request volume.
- The examples consume the context window, competing directly with the actual input and with any retrieved material (Module 11) — so the method scales badly exactly where you also want retrieval.
- It does not scale with data. Twenty examples help; two thousand do not fit in the context, so the method cannot make use of data you actually have, which is the decisive argument for fine-tuning once you have collected any.
- And the sensitivity makes evaluation harder — a result obtained from one prompt is a result about that prompt, so you must report variance across several phrasings and orderings to claim anything about the method.
- Which is a specific instance of Module 10 §1's single-number problem. 'A result from one prompt is a result about that prompt' is the discipline to adopt, and reporting across several prompts is cheap and almost never done — which makes a great deal of published prompting comparison weaker than it appears.

4 Instruction and preference tuning

INSTRUCTION TUNING ALIGNS BEHAVIOUR WITH THE TASK FRAMING, AND PREFERENCE TRAINING WITH HUMAN JUDGEMENT

Instruction tuning is supervised fine-tuning on (instruction, response) pairs across a great many tasks — which teaches the model to treat an instruction as *something to follow* rather than as *text to continue*, and that reframing is most of what distinguishes a usable assistant from a raw language model.

Preference training then optimises against human comparisons — either a reward model fitted to preference data followed by reinforcement learning (CSCE 642's machinery), or a direct preference objective that skips the explicit reward model and optimises the comparison directly.

And what this changes is behaviour rather than knowledge. The model does not learn new facts at this stage — the data volume is far too small — **it learns which of its possible continuations to actually produce**, which is a selection among existing capabilities.

Which is worth being clear about, because it has a direct consequence: preference training makes outputs more acceptable to raters, and acceptability is not accuracy — a confident, fluent, well-formatted wrong answer rates well, and that is Module 12 §3's mechanism rather than an incidental flaw.

- **No training data** → **prompting**, with the variance across phrasings measured and reported (§3).
- **Tens of examples** → **few-shot prompting, or a linear probe** if you can extract features (§1).
- **Hundreds to a few thousand** → **parameter-efficient fine-tuning** (§2), **which is the usual answer** and is cheap enough to try several configurations.
- **Tens of thousands of examples and a real domain shift** → **full fine-tuning**, if the compute is available and the gain over the parameter-efficient version is measured rather than assumed.
- **And a genuinely different domain with abundant raw text** → **continued pretraining first**, then one of the above on top. **The middle row covers most real cases: parameter-efficient fine-tuning on a few thousand examples is the modal correct answer**, and it is cheap enough to do several times with different data, which is usually a better use of the budget than one larger run.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Hu et al. — LoRA (free)	§2 in the original — including the rank ablations, which are the evidence for the low-rank hypothesis.
Min et al. — Rethinking the Role of Demonstrations (free)	§3's format-versus-mapping result — the random-label experiment, which is the one to know.
Ouyang et al. — Training language models to follow instructions (free)	§4's two stages , with the preference pipeline described concretely.

Resource	What it covers for this module
Rafailov et al. — Direct Preference Optimization (free)	§4's simpler alternative — the same objective without the separate reward model.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Run a linear probe** on frozen features for your task, and record the score.
- 2 **Try a small nonlinear head** and say what the difference tells you.
- 3 **Implement LoRA** on a small model and count the trainable parameters.
- 4 **Vary the rank** and plot quality against it.
- 5 **Merge the adapter into the base weights** and confirm the outputs match.
- 6 **Run the same few-shot task with five example orderings** and report the variance.
- 7 **Replace the few-shot labels with random ones** and measure the drop.
- 8 **Compute the per-request cost** of few-shot prompting versus fine-tuning at your expected volume.
- 9 **Compare a base and an instruction-tuned model** on the same instruction.
- 10 **Choose a method for Project 2** and justify it on data volume and constraints.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name six adaptation methods and what each changes.
- 2 Why run the linear probe first, and what does its failure tell you?
- 3 Explain LoRA and the hypothesis it rests on.
- 4 What three things does LoRA buy?
- 5 Why is 'in-context learning' a misleading name?
- 6 What do few-shot examples mostly contribute, and what is the evidence?
- 7 Give four limits of in-context learning.
- 8 What discipline should you adopt about prompt results?
- 9 What do instruction tuning and preference training each change?
- 10 Why is acceptability not accuracy?